Université Sultan Moulay Slimane
Ecole Supérieure de Technologie
- Beni Mellal -

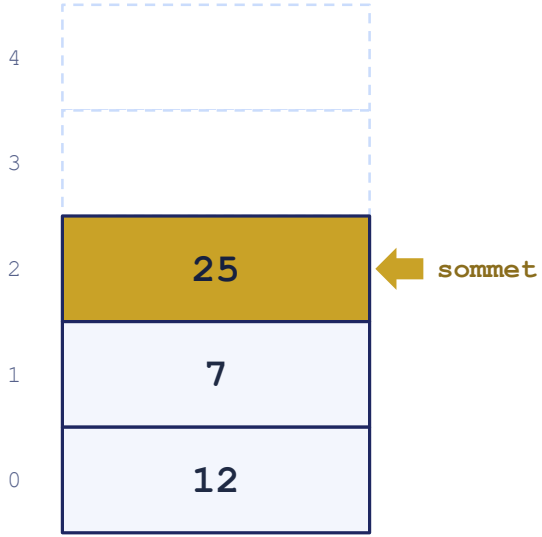
STRUCTURES DE DONNÉES

Chapitre 1 : Piles et Files Statiques

Filière : DUT GI
Semestre : S3
Année Universitaire : 2026/2027

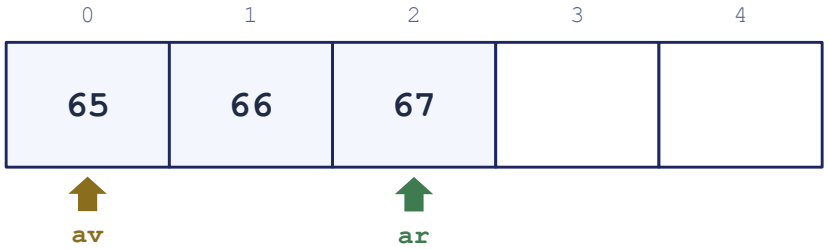
Pr. M. OUTANOUTE

La pile — dernier arrivé, premier servi



Pile de copies à corriger : la dernière déposée est la première corrigée — LIFO.

La file — premier arrivé, premier servi



Guichet d'inscription : le premier étudiant arrivé est le premier servi — FIFO.

La représentation statique : un tableau, un repère

Pile :

- ✓ Un tableau à taille fixe;
- ✓ Un seul repère : **sommet**.

File :

- ✓ Un tableau à taille fixe;
- ✓ Deux repères : **av** et **ar** (avant/arrière).

sommet, **av et ar** sont des indices — réservés aux structures statiques de ce chapitre.



Ce que ce chapitre va résoudre

Pile : comment savoir si elle est vide ou pleine ?

File : comment éviter le gaspillage d'espace, sans payer le coût d'un décalage ?

La réponse pour la file passe par TROIS versions successives.

PARTIE A — PILE

Déclaration, et les tests vide/pleine

```c
#define MAX 50

typedef struct {
    int elements[MAX];
    int sommet;
} Pile;

void initPile(Pile *p) { p->sommet = -1; }
int pileVide(Pile p)   { return (p.sommet == -1); }
int pilePleine(Pile p) { return (p.sommet == MAX - 1); }
```

sommet part de **-1**, pas **0** — les indices de tableau commencent à 0 en C.

PARTIE A — PILE

empiler

1) Si la pile est pleine, signaler l'erreur et ne rien faire.
2) Sinon, avancer sommet, puis écrire la valeur à ce nouvel emplacement.

```
void empiler(Pile *p, int valeur) {
    if (pilePleine(*p)) {
        printf("Pile pleine\n");
        return;
    }
    p->sommet++;
    p->elements[p->sommet] = valeur;
}
```

Ordre à respecter : ***sommet++*** *avant d'écrire — la nouvelle case, c'est là où* ***sommet*** *pointe désormais.*

PARTIE A — PILE

depiler

1) Si la pile est vide, signaler l'erreur et retourner -1.
2) Sinon, lire la valeur à **sommet**, reculer **sommet**, puis retourner la valeur lue.

```
int depiler(Pile *p) {
    if (pileVide(*p)) {
        return -1;
    }
    int valeur = p->elements[p->sommet];
    p->sommet--;
    return valeur;
}
```

Point de rigueur : sans le test **pileVide**, **elements[sommet]** avec **sommet=-1** est un accès mémoire invalide.

PARTIE B — FILE

Version 1 — la file ordinaire

```
typedef struct {
    int elements[MAX];
    int av;
    int ar;
} File;

void initFile(File *f) {
   f->av = -1;
   f->ar = -1;
}
int fileVide(File f)   { return (f.av == -1); }
int filePleine(File f) { return (f.ar == MAX - 1); }
```

Version 1 — enfiler/defiler, et le défaut

```
void enfiler(File *f, int valeur) {
    if (filePleine(*f)) { printf("File pleine\n"); return; }
    if (fileVide(*f)) { f->av = 0; }
    f->ar++;
    f->elements[f->ar] = valeur;
}

int defiler(File *f) {
    if (fileVide(*f)) { return -1; }
    int valeur = f->elements[f->av];
    f->av++;
    if (f->av > f->ar) { f->av = -1; f->ar = -1; }
    return valeur;
}
```

Le défaut : **ar** ne redescend jamais.

Exemple : MAX=5, on enfile 5 éléments puis défile 2 — enfiler refuse alors qu'il reste 2 cases libres en tête.

PARTIE B — FILE

Version 2 — la file par décalage

```
typedef struct {
    int elements[MAX];
    int ar;    /* nombre d'elements actuels */
} File;

void initFile(File *f) { f->ar = 0; }
int fileVide(File f)   { return (f.ar == 0); }
int filePleine(File f) { return (f.ar == MAX); }
```

Un seul repère (**ar**) : **av** disparaît, la tête est toujours à l'indice 0. **ar** devient un compteur d'éléments.

PARTIE B — FILE

Version 2 — enfiler/defiler, le gain et le coût

```c
void enfiler(File *f, int valeur) {
    if (filePleine(*f)) { printf("File pleine\n"); return; }
    f->elements[f->ar] = valeur;
    f->ar++;
}

int defiler(File *f) {
    if (fileVide(*f)) { return -1; }
    int valeur = f->elements[0];
    for (int i = 0; i < f->ar - 1; i++) {
        f->elements[i] = f->elements[i + 1];
    }
    f->ar--;
    return valeur;
}
```

Le gaspillage disparaît — mais **defiler** déplace tous les éléments restants à chaque appel.

PARTIE B — FILE

Version 3 — la file circulaire

```
typedef struct {
    int elements[MAX];
    int av;
    int ar;
} File;

void initFile(File *f) { f->av = -1; f->ar = -1;}
int fileVide(File f)   { return (f.av == -1); }
int filePleine(File f) { return ((f.ar + 1) % MAX == f.av); }
```

Le piège : **av == ar** ne suffit pas à distinguer une file vide d'une file à un seul élément.

Solution : une file vide a toujours **av = -1** — garanti par deux transitions particulières.

PARTIE B — FILE

Version 3 — enfiler : la transition « était vide »

1) Si la file est pleine, signaler l'erreur.
2) Avancer **ar** de façon circulaire : **ar = (ar + 1) % MAX**.
3) Si la file était vide avant cet ajout, **av** rejoint **ar**.
4) Écrire la valeur à l'emplacement **ar**.

```
void enfiler(File *f, int valeur) {
    if (filePleine(*f)) { return; }
    f->ar = (f->ar + 1) % MAX;
    if (fileVide(*f)) { f->av = f->ar; }
    f->elements[f->ar] = valeur;
}
```

PARTIE B — FILE

Version 3 — defiler : la transition « devient vide »

1) Si la file est vide, retourner -1.
2) Lire la valeur à **av**.
3) Si **av** et **ar** désignent le même élément (le dernier), les remettre tous deux à **-1**.
4) Sinon, avancer **av** de façon circulaire.

```
int defiler(File *f) {
    if (fileVide(*f)) { return -1; }
    int valeur = f->elements[f->av];
    if (f->av == f->ar) {   // la file contient un seul élément
        f->av = -1; f->ar = -1;  // initialisation de la file
    } else {
        f->av = (f->av + 1) % MAX;
    }
    return valeur;
}
```

Les trois versions — le bilan

	Ordinaire	Décalage	Circulaire
defiler	Immédiat	Coûteux (parcours)	Immédiat
Espace	Gaspillé	Récupéré	Récupéré
Variables	`av, ar`	`ar` seul	`av, ar`
Piège	**ar** ne redescend jamais	Oublier le décalage	Gérer les 2 transitions

La version Circulaire : cumule les deux avantages.

Exercice d’application

1) Une pile de copies. Écrire **empilerCopie** et **corrigerCopie**.
2) Dérouler 3 empilements (12, 7, 25) puis 2 corrections.
3) Une file circulaire de taille 5 (guichet). Écrire **enregistrerArrivee** et **guichetSuivant**.
4) Dérouler 4 arrivées puis 2 départs, avec **av** et **ar**.

Question 1 — le code

```
void empilerCopie(Pile *p, int numero) {
    empiler(p, numero);
}

void corrigerCopie(Pile *p) {
    int numero = depiler(p);
    if (numero != -1) {
        printf("Correction de la copie %d\n", numero);
    }
}
```

Question 2 — la trace de la pile

Étape	Action	Contenu de la pile (bas→haut)	sommet
1	`empiler(12)`	`[12]`	0
2	`empiler(7)`	`[12, 7]`	1
3	`empiler(25)`	`[12, 7, 25]`	2
4	**`corriger → 25`**	`[12, 7]`	1
5	**`corriger → 7`**	`[12]`	0

Etape 4: La copie 25, dernière empilée, est la première corrigée — LIFO.

Question 3 — la file circulaire du guichet

```
void enregistrerArrivee(File *f, int numero) {
    enfiler(f, numero);
}

void guichetSuivant(File *f) {
    int numero = defiler(f);
    if (numero != -1) {
        printf("Guichet : etudiant %d servi\n", numero);
    }
}
```

Question 4 — la trace de la file (MAX = 5)

Étape	Action	av	ar
1	`enfiler(65)`	0	0
2	`enfiler(66)`	0	1
3	`enfiler(67)`	0	2
4	`enfiler(68)`	0	3
5	**`defiler() → 65`**	**1**	3
6	**`defiler() → 66`**	**2**	3

Etape 5 : numéro 65, premier arrivé, est le premier à quitter la file — FIFO.

Récapitulatif

- Pile : **empiler** avance **sommet** avant d'écrire ; **depiler** lit avant de reculer **sommet**.
- Discipline systématique : tester **pileVide** avant un retrait, tester **pilePleine** avant un ajout.
- File ordinaire : **av**/**ar** incrémentent seulement — **ar** ne redescend jamais, gaspillage.
- File avec décalage : un seul repère **ar** — corrige le gaspillage, coûte un parcours à chaque **defiler**.
- File circulaire : **av**/**ar** avec % MAX — les deux transitions à gérer.

Exercice 1 : Test de l'équilibre des parenthèses

Écrire une fonction ***parenthesesEquilibrees*** qui reçoit une chaîne de caractères contenant les parenthèses **(** et **)**, et retourne **1** si les parenthèses sont bien équilibrées, **0** sinon, **en utilisant une pile**.

Indications :

- ✓ *empiler à chaque **(** rencontrée ;*
- ✓ *dépiler à chaque **)** rencontrée ;*
- ✓ *si la pile est vide au moment de dépiler, ou si elle n'est pas vide à la fin du parcours, les parenthèses ne sont pas équilibrées.*

Exercice 2 : Récupération du maximum d'une file

On considère une file circulaire d'entiers déclarée globalement de la manière suivante: un tableau des entiers (**F**) et deux variables représentant l'avant (**av**) et l'arrière (**ar**) de la file.

On veut ***récupérer le maximum de la file*** en respectant son principe de fonctionnement (FIFO), les fonctions à rédiger sont:

- initFile(); int fileVide(); int filePleine();
- void enfiler(int); int defiler();
- int recupererMax();
- void main()